NOSQL DATABASE

ASSIGNMENT 04

Questions and Answers

1. Explain the concept of Column-Oriented NoSQL Databases. How does the column-family model differ from the traditional relational table model? Discuss the advantages and limitations of using column-oriented databases for large-scale applications.

Answer:

A Column-Oriented NoSQL Database stores data using column families instead of traditional relational tables. It is designed to handle very large amounts of distributed data.

Difference from RDBMS:

- RDBMS stores data in fixed tables containing rows and columns.
- Column-family databases group related columns into column families.
- NoSQL databases have a more flexible schema.
- They are designed for horizontal scaling across multiple servers.

Advantages:

1. Handles huge amounts of data.
2. Provides high scalability.
3. Provides high availability and fault tolerance.
4. Supports flexible data structures.
5. Provides good performance for large distributed applications.

Limitations:

1. Joins are generally not supported like in RDBMS.
2. Data modeling requires careful planning.
3. Some NoSQL systems provide weaker consistency.
4. They are not suitable for every transaction-based application.

2. Compare Apache HBase and Apache Cassandra with respect to data model, architecture, consistency, availability, scalability, fault tolerance, query processing, and suitable applications. Which one would you choose for a highly distributed application and why?

Answer:

Comparison:

HBase — Column-family data model; Master/Region Server architecture; strong consistency; high availability; horizontal scalability; HDFS replication; HBase API/Phoenix; suitable for Hadoop and Big Data.

Cassandra — Wide-column data model; peer-to-peer architecture; tunable consistency; very high availability; horizontal scalability; data replication; CQL; suitable for real-time distributed applications.

For a highly distributed application, Cassandra would generally be preferred because it has a peer-to-peer architecture, high availability, horizontal scalability, and can continue working even when some nodes fail.

3. Explain the architecture of Apache HBase in detail. Describe the roles and interactionsof HMaster, RegionServer, ZooKeeper, HDFS, Regions, and MemStore during read and write operations. Draw a suitable architecture diagram.

Answer:

Apache HBase is a distributed NoSQL database built on HDFS.

Main Components:
HMaster: Manages the HBase cluster, assigns regions to RegionServers and performs load balancing.
RegionServer: Stores regions and handles read/write requests from clients.
ZooKeeper: Provides coordination between HBase servers and keeps track of server status.
HDFS: Provides distributed and fault-tolerant storage for HBase data.
Region: A large HBase table is divided into smaller parts called regions. Each region contains a range of row keys.
MemStore: A memory area where newly written data is temporarily stored before being written to HDFS.

Write: Client → RegionServer → MemStore → HDFS
Read: Client → RegionServer → MemStore/Cache/HDFS → Data

Architecture:
HBase Client → HMaster → RegionServers → Regions → HDFS; ZooKeeper provides cluster coordination.

4. Explain the Column-Family Data Store model with the help of a suitable example. Discuss concepts such as row key, column family, column, cell, timestamp, and version. How is this model different from rows and columns in an RDBMS?

Answer:

A Column-Family Data Store groups related columns into column families. It allows different rows to have different columns.

Example:
Row Key: S101
Personal: name = Rahul, age = 20
Academic: course = BCA, marks = 85
Contact: email = rahul@gmail.com

Concepts:
• Row Key — unique identifier of a row.
• Column Family — group of related columns.
• Column — individual data field.
• Cell — actual value stored in a column.
• Timestamp — records the time of a value.
• Version — different versions of a value can be maintained.

In an RDBMS, tables normally have a fixed structure. In a column-family database, rows can have different columns and the structure is more flexible.

5. A university needs to store billions of student activity records, where different students may have different attributes. Design a suitable HBase data model for this application and justify your choice of row key and column families.

Answer:

HBase is suitable because it can store billions of records and distribute them across multiple servers.

Row Key: StudentID#Timestamp
Example: S101#202610061030

Column Families: student, activity, device

Example:
student: name = Rahul, course = BCA activity: type
= Login, page = Dashboard device: ip =
192.168.1.10, browser = Chrome

Justification: StudentID identifies the student, Timestamp identifies the activity time, column families allow fexible attributes, and HBase can distribute large amounts of data across RegionServers.

6. Discuss consistency and transactions in HBase and Cassandra. Why do distributed NoSQL databases often make different trade-offs from traditional ACID-based relational databases? Explain how consistency can affect application design.

Answer:

Consistency means that users receive correct and updated data.

HBase: Provides strong consistency for individual row operations.
Cassandra: Provides tunable consistency. The application can choose levels such as ONE, QUORUM, and ALL.

Traditional RDBMS provides strong ACID transactions across tables. NoSQL databases generally use simpler transaction models to achieve better scalability and availability.

Distributed NoSQL systems often trade some traditional transaction features for high availability, scalability, performance, and fault tolerance.

Application designers must decide where strong consistency is required, which consistency level to use, how data will be replicated, and how conflicts will be handled.

7. Availability and fault tolerance are important characteristics of distributed databases. Explain how Apache Cassandra achieves high availability using its distributed architecture. Discuss the role of replication and consistency levels in maintaining availability.

Answer:

Availability means that the database remains accessible even when some servers fail. Cassandra uses a peer-to-peer architecture, so there is no single master node.

Replication: Cassandra stores copies of data on multiple nodes. If one node fails, another node can provide the data.

Consistency Levels:
• ONE — response from one replica.
• QUORUM — majority of replicas must respond.
• ALL — all replicas must respond.

Replication provides fault tolerance, while consistency levels allow the application to balance availability and consistency.

8. Explain the major query features of HBase and Cassandra. Compare HBase operations with Cassandra's CQL. Why is query and data modeling particularly important in column-family databases?

Answer:

HBase provides Get, Put, Delete, Scan, and Increment operations. It can also use Apache Phoenix for SQL-like queries.

Cassandra uses CQL (Cassandra Query Language). Common commands include CREATE, INSERT, SELECT, UPDATE, and DELETE.
Example: SELECT * FROM students WHERE id = 101;

Query patterns should be considered before designing the database because good partition keys improve speed, unnecessary scanning is avoided, data is distributed properly, hotspots are reduced, and queries become faster.

9. Scaling is one of the major reasons for using NoSQL databases. Explain how HBaseand Cassandra scale when data and user requests increase. Compare horizontal scaling, partitioning, replication, and load distribution in both systems.

Answer:

Both HBase and Cassandra use horizontal scaling.

HBase: Tables are divided into regions and distributed among RegionServers. HDFS provides distributed storage.
Cassandra: Data is divided into partitions and distributed among nodes. Replication provides fault tolerance.

Comparison:
• HBase partitioning — Regions; Cassandra — Partitions.
• HBase replication — HDFS; Cassandra — Cassandra replicas.
• HBase architecture — Master + RegionServers; Cassandra — Peer-to-peer.
• Load is distributed across RegionServers in HBase and nodes in Cassandra.

10. An organization wants to build a real-time event logging system that generates millions of events per minute. Explain why a column-oriented NoSQL database may be suitable for this application. Design a conceptual data model and discuss the required consistency, availability, and scaling strategy.

Answer:

A column-oriented NoSQL database is suitable because it can handle large amounts of data and high write rates.

Data Model:
Row Key: UserID#Timestamp#EventID event:
type, status, message user: userid, username
network: ipaddress, device

For event logging, very strong consistency may not always be required; high write availability is often more important. Replication should be used so data remains available if a node fails. Scaling can be achieved by adding nodes, partitioning data properly, distributing writes, and using replication.

11. Content Management Systems (CMS) and Blogging Platforms often contain articles, authors, tags, comments, metadata, and user activity. Explain how a column-family database can be used to design such systems. Discuss the challenges of handling varying and evolving data structures.

Answer:

A column-family database can store different CMS data using column families.

Example:
Row Key: Article101

article: title, content, author
metadata: category, date tags: tag1,
tag2

Comments can use ArticleID#CommentID as a key.

Challenges: Data structures may change, different articles may have different metadata, queries must be planned carefully, joins are not as easy as in RDBMS, and application logic may become more complex.

12. Counters are frequently used in distributed applications for tracking page views, likes, downloads, API requests, and user activity. Explain how counters can be implemented in Cassandra. What problems occur when counters are updated concurrently in a distributed environment?

Answer:

Counters maintain numerical values such as page views, likes, and downloads. For example, Page A may have 100 views, and each visit increments the counter.

When many nodes update the same counter simultaneously, concurrent updates can create synchronization and consistency challenges. Cassandra provides a counter data type designed for distributed counting. Proper data modeling should be used to reduce contention and improve performance.

12. Explain the concept of expiring data (TTL – Time To Live) in Cassandra/HBase.Consider a system that stores temporary user sessions and automatically removes them after 24 hours. Explain how TTL can be useful and discuss its impact on storage management and application design.

Answer:

TTL (Time To Live) is used to automatically expire data after a specified period. For example, a user session can be stored with a TTL of 24 hours.

Uses: Temporary sessions, cache data, temporary login information, and short-term data.

Advantages:
1. Automatically removes old data.
2. Reduces manual deletion.
3. Helps manage storage.
4. Useful for temporary information.

Expired data may not be physically removed immediately. Database maintenance and compaction eventually remove the expired data.

A company is developing a real-time analytics platform that must handle high writevolume, large datasets, geographically distributed users, and occasional node failures. Compare HBase and Cassandra for this scenario, and recommend one database. Justify your answer using architecture, consistency, availability, scalability, and query requirements.

Answer:

Both HBase and Cassandra can handle large distributed datasets.

HBase: Good for Big Data, strong consistency, Hadoop ecosystem integration, and analytical workloads.
Cassandra: Peer-to-peer architecture, very high availability, horizontal scalability, tunable consistency, high write performance, and strong fault tolerance.

Recommendation: Cassandra would generally be preferred because the platform requires high write performance, high availability, geographically distributed operation, horizontal scalability, and fault tolerance.

12. Column-oriented NoSQL databases provide scalability and flexibility, but they requirecareful data modeling. Critically analyze this statement using HBase and Cassandra as examples. Discuss how poor choices of partition keys, row keys, column families, replication, and query patterns can negatively affect system performance.

Answer:

Column-oriented NoSQL databases provide high scalability and flexible data storage, but careful data modeling is essential.

Partition Key: A poor partition key can put too much data on one node and create a hotspot.
Row Key: In HBase, a poorly designed row key can send too many writes to one region, causing uneven load.
Column Families: Poorly designed column families can increase storage and processing overhead.
Replication: Higher replication improves availability but increases storage, network traffic, and write overhead.
Query Patterns: Poor query design can cause full table scans, slow reads, and high resource usage.

Conclusion: Proper selection of row keys, partition keys, column families, replication settings, and query patterns is essential to achieve good performance, scalability, and availability in HBase and Cassandra.